

PACT: Contract-Verified Receding-Horizon Planning for Compute-Bounded Tool Agents

Pu Yang

Independent Researcher
npuyangpu@gmail.com

Abstract

Tool agents often let one language model both interpret intent and authorize external effects. We instead present PACT, a contract-guided Track 2 agent that treats its Cerebras-hosted model as an *untrusted semantic compiler*. The model emits a compact strict-schema wire plan; a deterministic kernel then performs strict decoding, typed-DAG validation, live Draft 2020–12 JSON Schema checking, and evidence-aware execution. PACT exposes at most one outstanding action, recompiles at semantic data boundaries, scopes confirmation to an exact operation–argument–capability–snapshot triple, and carries successful action obligations into the evidence conditions of a typed terminal outcome. One verification-guided repair is permitted; plans containing an ACT undergo one separate second-pass stochastic policy audit and deterministic re-verification. Provider retries obey finite retry and cumulative-wait budgets, while final A2A text responses report mutually exclusive visible completion and thinking-token metrics. Guarantees cover local structure, contracts, transitions, and evidence—not semantic interpretation. The release suite exceeds 350 tests; a historical public smoke trajectory is reported only as engineering evidence, not a performance estimate.

1 Introduction

CAR-bench evaluates whether tool agents act consistently and respect limits under ambiguous or incomplete information [Kirmayr *et al.*, 2026]. Through the Agent2Agent (A2A) protocol [A2A Protocol Working Group, 2026], the evaluator supplies policy text, messages, live tool declarations, and tool results; it alone executes external operations. Track 2 additionally rewards useful behavior under a bounded Cerebras inference route. A ReAct-style loop [Yao *et al.*, 2023] is flexible, but the same stochastic component commonly interprets the request, constructs arguments, decides that preconditions hold, and declares success.

PACT separates those responsibilities. The LLM translates language into a small obligation program, but has no execution authority. A local monitor admits the program

only after checking its syntax, graph, current capability contracts, resource bounds, confirmations, and evidence dependencies. This resembles proof-carrying code’s separation of an untrusted producer from a small checker [Necula, 1997] and runtime verification’s monitoring of concrete executions [Leucker and Schallhart, 2009]. This is an architectural analogy, not a claim that natural-language meaning is formally proved: the accepted artifact is only a locally checkable structural/provenance certificate.

Our contributions are: (i) a provider-compatible, strict wire format that decouples structured generation from dynamic tool schemas; (ii) a typed PlanIR and receding-horizon obligation runtime with evidence-gated typed outcomes, individually consumed confirmation scopes, and contract revalidation at emission; and (iii) a precise assurance boundary plus reproducible failure-oriented tests and compute telemetry. The trusted kernel is domain-agnostic: it contains no task IDs, benchmark utterances, or automotive workflow branches.

Positioning. PACT is neither a direct model-to-tool interface nor a catalog of benchmark-shaped workflows. Direct tool calling constrains the surface form of an action, but syntactically valid JSON does not establish that the named operation is still available, its arguments satisfy a changing recursive schema, a confirmation applies to this exact effect, or a result supports the claimed terminal outcome. Empirical work on structured generation similarly finds a gap between nominal schema conformance and reliable generation across real schemas [Geng *et al.*, 2025]. PACT therefore uses two languages: a small provider-facing wire language optimized for constrained decoding, and a richer local PlanIR checked against the concrete runtime state. Conversely, unlike a hand-authored finite-state policy, the PlanIR is synthesized anew from the current policy, request, evidence, and capabilities. Selectivity occurs at each verified plan horizon: the model proposes semantics, while the kernel owns only transitions that are structurally determined by the accepted graph.

2 PACT Architecture

2.1 State and Untrusted Compilation

For one active goal in A2A context q , PACT maintains

$$S_t^q = (P, g, H_t, T_t, E_t, G_t, p_t, O_t, C_t, M_t), \quad (1)$$

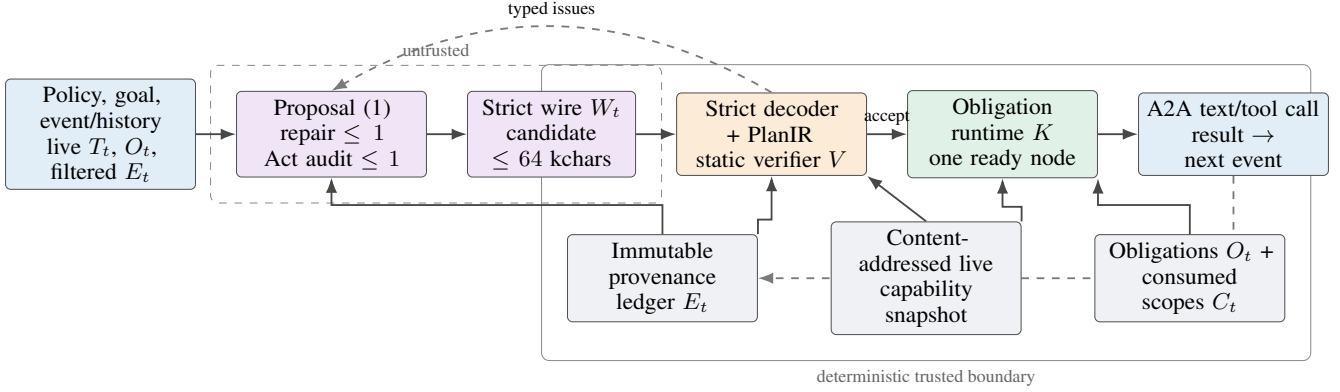


Figure 1: PACT separates semantic proposal from execution authority. Each compile uses one proposal, at most one repair, and at most one Act-only audit ($N_j \leq 3$ sequential completions). Proposal and repair candidates are verified before any Act-only audit, whose replacement candidate is reverified. The runtime emits one obligation at a time. Results may enter bounded compiler history, but only the provenance ledger may discharge proof obligations; the live capability snapshot is checked at admission and call emission.

where P is the evaluator-supplied policy, g the current goal, H_t the bounded compiler-visible suffix of a longer context conversation, T_t the live tool contracts, E_t a per-goal append-only evidence ledger, G_t the accepted plan, p_t its single pending action, O_t carried completion obligations, C_t consumed confirmation scopes, and M_t accumulated inference metrics. Each exact scope in C_t is projected out of future compiler inputs after use. Context cancellation deletes this state while retaining a stable per-context lock to serialize concurrent turns.

The semantic compiler C_θ receives P in a separate trusted system partition and the goal, recent events, T_t , and a typed projection of E_t as task state. It proposes a wire plan

$$W_t = C_\theta(P, g, H_t, T_t, \text{view}_{C_t}(E_t), O_t). \quad (2)$$

The provider schema is strict: every object forbids extra properties, and its canonical serialization is capped at 5,000 characters to match the Cerebras *response-schema* contract [Cerebras Systems, 2026]; this is not the generated candidate limit, which is independently 64,000 characters. Dynamic arguments cannot be represented as an unrestricted object under that contract, so each operation node carries an `arguments_json` string. Locally, PACT rejects duplicate keys and non-finite numbers, decodes that string to one JSON object, and validates it against the actual live schema. Thus provider constrained decoding is an early filter, not the authority. If local checking rejects the first proposal, one repair prompt contains value-redacted typed issues; a second rejection terminates safely. If the verified candidate contains any ACT, a separate second-pass model call reviews the complete plan against policy, goal, evidence, capabilities, and argument provenance. Its full corrected candidate is decoded and verified again; failure rejects the plan. This audit is a stochastic cross-check, not a semantic proof, and is skipped for action-free plans. The submitted default is `gpt-oss-120b`, medium reasoning, and an 8,192-token output budget. A provider `length_finish` is recorded as typed truncation and is not repaired by repeating the same call; empty content is likewise a typed invalid candidate and never reaches PlanIR.

2.2 Typed PlanIR and Live Verification

After decoding, a plan is a bounded directed acyclic graph with nodes from

$$\mathcal{N} = \{\text{OBSERVE, ASK, CONFIRM, ACT, RESPOND}\}.$$

Node IDs and evidence keys are unique; dependencies must exist and be acyclic; every node must reach exactly one terminal RESPOND. The default limits are 20 nodes and dependency depth 12. OBSERVE and ACT contain a live operation, constant JSON arguments, and a success key. ASK and CONFIRM wait for a correlated user event. CONFIRM also names the exact descendant action nodes it authorizes. RESPOND has one typed outcome—completed, answered, refused, declined, or fail-safe—and an explicit evidence set.

From T_t , PACT constructs a canonical, content-addressed capability snapshot. Each operation’s argument schema is checked and executed with a Draft 2020–12 validator [JSON Schema, 2022]; nested objects, arrays, formats, numeric constraints, combinators, and additional-property rules therefore remain in the trusted boundary. Duplicate or malformed capability declarations fail closed. If trusted metadata labels an operation as `observe` or `act`, node kind must agree. PACT does not infer effects from operation names; absent metadata is represented as `unknown` and receives no kind-level guarantee.

2.3 Receding-Horizon Obligation Execution

Let $\sigma_t(n) \in \{\text{pending, waiting, satisfied, failed}\}$. The kernel selects the first topologically ready node

$$R_t = \{n \mid \sigma_t(n) = \text{pending}, \forall d \in \text{dep}(n), \sigma_t(d) = \text{satisfied}\}, \quad (3)$$

$$n_t = \min_{\prec} R_t. \quad (4)$$

It emits only n_t and then waits. Immediately before an external call, a fresh snapshot must have the same digest, contain the operation, preserve any known effect label, and accept a deep-copied argument object. Ask and observation results are semantic data boundaries that trigger a new short-horizon

compile; so does a qualified confirmation that revises scope. By contrast, an exact unqualified confirmation and a matched ACT result are control events: positive confirmation and action success advance the already-verified immutable suffix without an LLM call; rejection or failure terminates deterministically. A capability change detected while releasing a pending confirmation triggers replanning without inherited scope; a change detected at external-call emission fails closed. Any successful action key is also placed in O_t , so a later horizon cannot silently forget a partially completed request.

Records carry applicable provenance fields: source and status; event, call, and node IDs; operation, context, and plan; argument/schema digests; and authorizations. Values are isolated through a JSON round trip on write and read, preventing nested caller mutation from rewriting history. Fingerprinted event IDs make identical replay idempotent; conflicting reuse, out-of-order input, and explicit external failure cannot satisfy that plan.

For runtime events and confirmations, `schema_digest` stores SHA-256 of the entire canonical capability snapshot—contracts and trusted annotations—not merely one operation’s parameter schema.

Confirmation is capability-conditional and scoped. For an operation marked confirmation-required, the verifier accepts either an ancestor CONFIRM authorizing that action node or a prior true confirmation record matching

$$(D(T_t), \text{operation}, \text{SHA256}(\text{arguments})). \quad (5)$$

Here $D(T_t)$ denotes that full snapshot digest. After matching action success, the adapter records the confirmation key, operation, and argument digest in C_t , then projects that scope out of future compiler evidence. It cannot authorize another call in a future horizon. One confirmation record may explicitly contain several action scopes in the accepted immutable plan; its other scopes remain independent until individually consumed.

3 Assurance and Compute Bounds

We distinguish properties checked by code from assumptions about its inputs. Suppose the adapter is the only emission path, the live capability declaration is a valid schema, and hashing/JSON validation behave as specified.

Capability and schema confinement. If PACT emits external call $a_t = (o, x)$, then $o \in T_t$ and $x \models \text{schema}_{T_t}(o)$. The verifier establishes this for the accepted immutable plan; the emission guard repeats both membership and validation on a fresh digest-matched snapshot. Mutation or revocation between the two checks yields no call.

Evidence-closed completion. For a typed `completed` response, PlanIR requires at least one action or carried evidence, the verifier requires every current action-success key and every key in O_t in the terminal evidence set, and carried keys must be successful external action records. At emission, every cited record must be external with status `success`. Hence no failed, user-authored, missing, or planned event can justify the `completed` outcome category. The semantic correctness of response prose remains model-produced. `answered` is action-free and cites observations selected by local lowering;

`refused` calls nothing; `declined` requires a negative confirmation. Emergency `fail_safe` may lack evidence after protocol/compiler failure, but never claims completion.

Transition confinement. Within the kernel, at most one node is waiting. Only a type-compatible event for that internal call can change it to satisfied. External events additionally match context, plan, node, operation, argument digest, schema digest, and a nonempty evaluator call ID. This establishes replay/idempotency properties *after* the A2A adapter has correlated the sole returned result; the protocol limitation below is material.

Failure non-escalation (invariant argument). Proceed by induction over processed events. Initially E_0 is empty and no node is satisfied. Registration of an unmatched event changes only its audit disposition. A matched input event can satisfy only the pending ASK/CONFIRM. A matched external event creates success evidence only when its explicit status is `success`; failure instead creates a distinct failure key and marks the node failed. Duplicate and conflicting event IDs do not change the first stored record. Therefore none of these cases can manufacture a successful action record. The completion guard consults only such records, preserving the invariant.

Obligation monotonicity (invariant argument). Within one goal, O_t grows after each satisfied ACT and is cleared only at a terminal or explicit safe termination. Every new compiler request receives O_t . The verifier rejects a carried key that is not a successful external action and rejects a completed terminal omitting any member of O_t . Thus, by induction over horizons, a reported completion accounts for all earlier successful nodes classified as ACT. If a later obligation fails, the adapter instead emits partial-failure text.

Scope-limited confirmation (invariant argument). For an annotated capability, admission requires either an authorizing ancestor in the same immutable plan or a true ledger record whose schema, operation, and argument digests match. After a matching action succeeds, the adapter removes that individual authorization through C_t from the compiler-visible evidence projection. A record may hold multiple action scopes, but each exact scope is consumed independently; changing any tuple component makes it unusable.

Bounded inference and telemetry. For compiler request j within one Track 2 decision step, with repair indicator $r_j \in \{0, 1\}$ and Act-audit indicator $s_j \in \{0, 1\}$,

$$N_j = 1 + r_j + s_j \leq 3. \quad (6)$$

The audit is spent only after a locally accepted plan contains an ACT; an action-free answer uses $s_j = 0$. The audited candidate again crosses the full decoder/verifier boundary. Input length, conversation count, evidence count, candidate size, graph size, and graph depth have independent local ceilings. Provider rate-limit handling uses a finite retry limit $R_{\max}$ and cumulative wait limit $W_{\max}$, including proactive pacing and reactive header-aware backoff; exhaustion raises a typed error rather than waiting indefinitely. Across tool turns, provider usage is accumulated until the next text response and exported as A2A `turn_metrics`. Competition-accounted usage is

$$B(q) = \sum_{j \in q} (p_j + c_j^{\text{vis}} + h_j), \quad (7)$$

where c_j^{vis} excludes the reasoning tokens h_j . Cerebras reports reasoning inside its completion total, so the adapter subtracts h_j before populating `A2A completion.tokens`; completion and `thinking.tokens` are therefore mutually exclusive. Call count, mean call time, model, cost, passes, and quota wait are also reported; safe errors retain usage from failed verification attempts. Equation 7 is evaluator-facing telemetry, not a local hard cap on task-level trajectory length; PACT instead enforces per-input, per-candidate, per-call, and per-compilation bounds.

Liveness boundary. For a stable accepted DAG, each matched control event changes one waiting node, so its deterministic suffix reaches a terminal or Ask/Observe boundary within $|\mathcal{N}|$ emissions. A missing user, exhausted provider, malformed result, or revoked capability can stop progress safely; semantic replanning can also continue indefinitely. Bounds therefore apply per compilation and accepted horizon, not to an open conversation.

4 Engineering Evaluation and Limits

We evaluate enforceable mechanisms, not hidden outcomes. On 17 July 2026, 356 tests plus three subtests covered PlanIR, verification, compilation, ledger/runtime transitions, A2A, full JSON Schema, and bounded provider retries. Adversarial cases include cycles, revocation, malformed/replayed results, deep mutation, confirmation reuse, forgotten actions, exhausted repair, cancellation, and safe-error metrics. Generic non-automotive schemas exercise the same kernel. These deterministic `uv run pytest -q` checks do not estimate benchmark reward or three-trial consistency.

Historical single-task smoke. An earlier low/4,096 public `base_0` run earned all five subrewards. Seven calls used 97,922 prompt, 1,940 recomputed visible-completion, and 2,032 thinking tokens (101,894 total), with 360,753.4 ms quota wait. It predates the final audit/control suffix and is not a performance estimate. A separate truncation canary motivated the 8,192 ceiling and typed fail-closed handling.

The final closure also passes Ruff and mypy over 18 release files. The public, digest-pinned `linux/amd64` image completed README Option C with client exit code zero. This establishes packaging/A2A compatibility, not live performance; the personal development quota expired during that smoke run.

Operational reproducibility. The selective image runs unprivileged on pinned bases; secrets enter only at runtime, while model/retry settings remain environment-configurable. Compiler results expose hashes and value-redacted issues, but raw candidates and the process-local ledger are not persisted. Stable context locks prevent a cancellation replacement-lock race.

Local checks do not prove natural-language interpretation; the second-pass audit is stochastic. An operation with unknown effect metadata may be either OBSERVE or ACT, so confirmation guarantees require an explicit annotation. A2A also exposes no agent nonce: with one outstanding call, PACT checks operation and nonempty evaluator ID, but cannot distinguish a replayed identical result cryptographically. Hashes are content addresses, not signatures.

PACT’s central result is architectural: stochastic inference proposes meaning, while a small deterministic layer owns capability admission, evidence-bearing state changes, and evidence conditions for typed terminal outcomes. This separation supports compute-bounded execution without benchmark-specific workflows and makes both the achieved guarantees and the remaining semantic uncertainty explicit.

References

- [A2A Protocol Working Group, 2026] A2A Protocol Working Group. Agent2Agent protocol specification, version 1.0.0. Linux Foundation, 2026. Accessed: 2026-07-17.
- [Cerebras Systems, 2026] Cerebras Systems. Structured outputs. Cerebras Inference Documentation, 2026. Accessed: 2026-07-17.
- [Geng *et al.*, 2025] Saibo Geng, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Generating structured outputs from language models: Benchmark and studies, 2025.
- [JSON Schema, 2022] JSON Schema. JSON Schema Core, draft 2020-12. Specification, 2022. Accessed: 2026-07-17.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, 2026. Association for Computational Linguistics.
- [Leucker and Schallhart, 2009] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [Necula, 1997] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023.